Trabalho Prático

Processamento de Linguagens

Grupo No. 15

No.27958 – Bruno Dantas
No.22840 – Gonçalo Correia
No.21143 – João Castro
No.21147 – André Ferreira

Licenciatura em Engenharia de Sistemas Informáticos
2º ano

Barcelos | maio, 2026

Índice

1. Introdução 3
2. Desenvolvimento 4
    2.1. Como foi desenvolvido 4
    2.2. Analisador Léxico 5
    2.3. Analisador Sintático 6
    2.4. Analisador Semântico 7
    2.5. Interpretador 8
3. Testes 9
4. Conclusão 10
5. Bibliografia 11

1. Introdução

Este trabalho prático tem como objetivo o desenvolvimento de um interpretador para a linguagem funcional simplificada LFun, no âmbito da unidade curricular de Processamento de Linguagens. A LFun é uma linguagem de programação funcional que suporta expressões aritméticas e booleanas, definições de variáveis e funções, estruturas condicionais e seleção de casos.

O interpretador foi desenvolvido em Python, recorrendo à biblioteca PLY (Python Lex-Yacc), que disponibiliza ferramentas para a construção de analisadores léxicos e sintáticos. O sistema processa ficheiros de texto com extensão .lf, interpreta os comandos neles contidos e apresenta os resultados no terminal.

O processo de desenvolvimento seguiu as seguintes fases:

- Especificação da gramática concreta da linguagem LFun;
- Desenvolvimento do analisador léxico com PLY (lex);
- Desenvolvimento do analisador sintático com PLY (yacc) e construção da AST;
- Implementação da análise semântica com tabela de símbolos e verificação de tipos;
- Desenvolvimento do interpretador para avaliação da AST e execução do programa.

2. Desenvolvimento

2.1. Como foi desenvolvido

O projeto foi desenvolvido em Python e organizado em seis ficheiros principais, cada um responsável por uma fase distinta do processo de interpretação:

- lexer.py — Analisador léxico: tokenização do código fonte;
- ast_nodes.py — Definição dos nós da Árvore de Sintaxe Abstrata (AST);
- grammar.py — Analisador sintático: construção da AST a partir dos tokens;
- semantic.py — Análise semântica: verificação de tipos e tabela de símbolos;
- interpreter.py — Interpretador: avaliação da AST e execução do programa;
- main.py — Ponto de entrada: lê o ficheiro .lf e coordena os restantes módulos.

Estes componentes trabalham de forma integrada e sequencial. O lexer transforma o texto de entrada numa sequência de tokens, que são depois processados pelo parser para construir a AST. A AST é então verificada pelo analisador semântico e, se não houver erros de tipos, é avaliada pelo interpretador, que produz os resultados no terminal.

2.2. Analisador Léxico

O analisador léxico, desenvolvido com a biblioteca PLY (ply.lex), tem como função identificar e classificar os elementos básicos da linguagem LFun, convertendo o texto de entrada numa sequência de tokens que representam as unidades mínimas significativas da linguagem.

Os tokens reconhecidos incluem:

- Palavras reservadas: let, fun, if, then, else, when, is, end, true, false, Int, Bool;
- Identificadores: nomes de variáveis e funções (ex: x, base, inc);
- Números inteiros (ex: 42, 10, 0);
- Operadores aritméticos: +, -, *, /;
- Operadores de comparação: ==, !=, <, >, <=, >=;
- Operadores booleanos: &&, ||, !;
- Símbolos especiais: :, ;, =, ->, (,), _;
- Comentários de linha (--) e de bloco ({- ... -}), que são ignorados pelo lexer.

As palavras reservadas são distinguidas dos identificadores comuns através de um dicionário de palavras reservadas, consultado na função t_ID. O lexer também trata

espaços em branco, tabulações e quebras de linha, ignorando os primeiros e atualizando o contador de linhas para os últimos, o que permite reportar erros com a linha correta.

Exemplo de tokens que estão no lexer:

```
47  #expressões regulares
48  t_ignore      = ' \t'
49  t_ARROW       = r'->'
50  t_EQ          = r'=='
51  t_NEQ         = r'!='
```

```
71  def t_COMMENT_LINE(t):
72      r'--[^\n]*'
73      pass
74
```

2.3. Analisador Sintático

O analisador sintático, desenvolvido com PLY (ply.yacc), recebe a sequência de tokens produzida pelo lexer e verifica se a estrutura sintática está de acordo com a gramática da linguagem LFun. Quando uma regra gramatical é satisfeita, é construído o nó correspondente da AST.

A gramática suporta as seguintes construções:

- Declarações de variáveis: let base : Int = 10;
- Assinaturas de funções: fun inc : Int -> Int;
- Definições de funções: let inc x = x + 1;
- Expressões aritméticas com precedência correta de operadores;
- Expressões booleanas com comparações e operadores lógicos;
- Condicional if/then/else;
- Seleção de casos when/is/end com padrões numéricos, booleanos e wildcard (_);
- Chamadas de funções: inc(5).

A precedência dos operadores foi definida explicitamente para garantir a avaliação correta das expressões, com * e / tendo maior prioridade do que + e -, e os operadores booleanos tendo menor prioridade do que os aritméticos:

```
9   precedence = (
10      ('left',  'OR'),
11      ('left',  'AND'),
12      ('right', 'NOT'),
13      ('left',  'EQ', 'NEQ'), #EQ = igual
14      ('left',  'LT', 'GT', 'LEQ', 'GEQ')
15      ('left',  'PLUS', 'MINUS'),
16      ('left',  'TIMES', 'DIVIDE'),
17      ('right', 'UMINUS'),
18  )
```

2.4. Analisador Semântico

O analisador semântico, implementado no ficheiro semantic.py, é responsável por verificar a coerência semântica do programa após a análise sintática. Para tal, mantém uma tabela de símbolos que regista o tipo de cada variável e função declarada.

As verificações realizadas incluem:

- Verificação de que o tipo declarado de uma variável corresponde ao tipo da expressão atribuída;
- Verificação de que funções têm assinatura declarada antes da definição;
- Verificação de que o corpo de uma função tem o tipo de retorno indicado na assinatura;
- Verificação de tipos nos operadores aritméticos (requerem Int);
- Verificação de tipos nos operadores de comparação (produzem Bool);
- Verificação de que a condição do if é do tipo Bool e que ambos os ramos têm o mesmo tipo;
- Verificação de que todos os casos do when produzem o mesmo tipo;
- Verificação de chamadas de funções (argumento com tipo correto).

Quando é detetado um erro semântico, é lançada uma exceção SemanticError com uma mensagem descritiva, e o programa termina sem chegar à fase de interpretação. Por exemplo (exemplo tirado do ficheiro 02_defenicoes.lf):

```
{- Este exemplo deve ser rejeitado pelo analisador semântico,
   porque a expressão definida tem tipo Bool, mas o identificador
   foi declarado com tipo Int. -}
let valor : Int = true;
```

```
[Semântico] Tipo errado em 'valor': declarado 'Int' mas a expressão é 'Bool'
PS C:\Universidade2025-26\2Semestre\PL\TrabalhoPratico\tp-15-1\source>
```

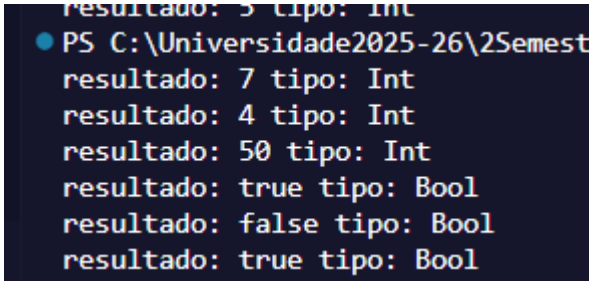
2.5. Interpretador

O interpretador, implementado no ficheiro interpreter.py, avalia a AST e produz os resultados do programa. Mantém um ambiente de execução que associa nomes a valores, e avalia cada expressão recursivamente.

O interpretador suporta:

- Avaliação de expressões aritméticas e booleanas;
- Acesso a variáveis previamente definidas;
- Execução de condicionais if/then/else;
- Avaliação de casos when, percorrendo os padrões até encontrar o primeiro que case;
- Chamadas de funções com closures, preservando o ambiente de definição da função.

Para cada declaração let com tipo, o interpretador imprime o resultado e o tipo no terminal, no formato definido pelo enunciado. O print abaixo serve como demonstração do resultado do ficheiro “01_arith_lf”:



3. Testes

Para testar o interpretador foram criados cinco ficheiros de entrada na pasta input/, cada um cobrindo uma secção da linguagem LFun descrita no enunciado.

01_arith.lf — Expressões aritméticas e booleanas:

```
-- Expressões aritméticas
let a1 : Int = 3 + 4;
let a2 : Int = 10 - 2 * 3;
let a3 : Int = (8 + 2) * 5;
```

```
-- Expressões booleanas
let b1 : Bool = 3 < 5;
let b2 : Bool = 10 == 7;
let b3 : Bool = (4 + 1) == 5;
```

02_definicoes.lf — Definições com let e uso de variáveis:

```
-- Secção B: Definições com let

let base : Int = 10;
let ativo : Bool = true;
let area : Int = 4 * 5;

-- Usar variáveis em expressões seguintes
let largura : Int = 8;
let altura : Int = 5;
let areaRect : Int = largura * altura;
let teste : Bool = altura < largura;
```

03_funcoes.lf — Funções e chamadas:

```
-- Secção C: Funções

fun inc : Int -> Int;
let inc x = x + 1;

fun dobro : Int -> Int;
let dobro n = n * 2;

fun ePositivo : Int -> Bool;
let ePositivo n = n > 0;
```

```
-- Testar as funções
let r1 : Int = inc(5);
let r2 : Int = dobro(7);
let r3 : Bool = ePositivo(3);
let r4 : Bool = ePositivo(-2);
```

04_condicional.lf — if/then/else e when:

Alguns que estão presentes no projeto:

```
-- Secção D: Condicional e seleção de casos

-- if/then/else
fun absoluto : Int -> Int;
let absoluto n = if n < 0 then 0 - n else n;

-- when com padrões numéricos
fun eZero : Int -> Bool;
let eZero n =
  when (n) is
    0 -> true;
    _ -> false;
  end;
```

05_comentarios.lf — Comentários de linha e de bloco:

```
-- Secção E: Comentários

-- Isto é um comentário de linha

{- Isto é um comentário
   de múltiplas linhas -}

let x : Int = 10; -- comentário no fim da linha

{- Este let está comentado e não deve ser processado
let y : Int = 20;
-}

let z : Int = 5;
```

Também foi testada a deteção de erros semânticos, confirmando que o analisador rejeita corretamente situações como a atribuição de um valor Bool a uma variável declarada como Int.

4. Conclusão

Este projeto permitiu consolidar os conhecimentos adquiridos na unidade curricular de Processamento de Linguagens, nomeadamente na construção de analisadores léxicos e sintáticos, na definição de gramáticas formais e na implementação de análise semântica.

O interpretador da linguagem LFun foi implementado com sucesso, suportando todas as funcionalidades descritas no enunciado: expressões aritméticas e booleanas, definições de variáveis e funções, condicionais if/then/else, seleção de casos with when/is/end e comentários de linha e de bloco.

A principal dificuldade encontrada foi a integração entre os diferentes módulos, em particular a gestão do lexer pelo parser e a resolução de conflitos shift/reduce na gramática. Estes problemas foram resolvidos através da definição explícita de precedências e da correta inicialização do lexer em cada chamada ao parser.

O projeto encontra-se funcional e cobre quase todos os requisitos base do enunciado, estando preparado para uma eventual extensão com novas funcionalidades como funções recursivas ou tipos adicionais.

5. Bibliografia

PLY (Python Lex-Yacc) — David Beazley, 2025.

Disponível em: https://www.dabeaz.com/ply/ply.html

Python Documentation — Python Software Foundation, 2025.

Disponível em: https://docs.python.org/3/

Slides das aulas de Processamento de Linguagens — IPCA, 2025/2026.